MCP/RAG System Architecture

AI-Assisted Infrastructure for NOAA's GFS Software Development Initiative

---

Technical Reference Document — Version 2.0.0

Environmental Information Branch
Environmental Modeling Center
National Oceanic and Atmospheric Administration

Terry.McGuinness@noaa.gov

with contributions from
Claude (Anthropic) — AI Pair Programming Assistant

January 6, 2026

Executive Summary

This document serves as the authoritative technical reference for the **Model Context Protocol (MCP) with Retrieval-Augmented Generation (RAG)** system developed by the Environmental Information Branch (EIB) to support NOAA's flagship Global Forecast System (GFS).

The system provides AI-assisted code analysis, compliance validation, and operational guidance for the Global Workflow—a complex multi-repository codebase spanning Fortran, Python, Bash, and C that produces operational weather forecasts for the National Weather Service.

Key Capabilities

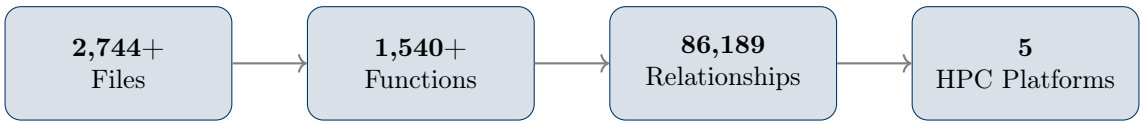
- ✓ 38 MCP tools for code analysis, semantic search, and compliance
- ✓ Hybrid retrieval: ChromaDB (vectors) + Neo4j (graph relationships)
- ✓ EE2 compliance validation against NCO production standards
- ✓ Multi-platform HPC guidance (Hera, WCOSS2, Orion, Hercules, Gaea)
- ✓ COTS LLM independence—works with Claude, GPT-4, Gemini, local models

Contents

1 Mission and Scope

1.1 Problem Statement

The Global Workflow represents one of NOAA's most complex software systems:



Traditional documentation and manual code review cannot scale to support:

- New developer onboarding to complex legacy codebases
- Cross-platform troubleshooting across HPC systems
- Compliance validation before NCO submission
- Understanding call relationships in 50+ submodules

1.2 Solution: AI-Assisted Operations

The MCP/RAG system provides AI assistants (VS Code Copilot, Claude Desktop, n8n) with structured access to the Global Workflow knowledge base:

Capability	Implementation	Example Query
Code Understanding	Neo4j call graphs	"What functions call `err_chk`?"
Compliance Checking	EE2 + SME annotations	"Is this script EE2 compliant?"
Operational Guidance	Platform-specific docs	"How do I run GFS on Hera?"
Dependency Analysis	Graph traversal	"What depends on this module?"

Table 1: AI-assisted capabilities provided by the MCP/RAG system

1.3 Design Principles

Key Insight

Four core principles guide the system architecture:

1. **COTS LLM Independence**: No vendor lock-in; works with any MCP-compatible client
2. **SPOT**: Single Point of Truth for each configuration domain
3. **MCP-First Policy**: AI uses MCP tools before shell commands
4. **Offline Capable**: Full functionality after initial setup

2 System Architecture

2.1 High-Level Architecture

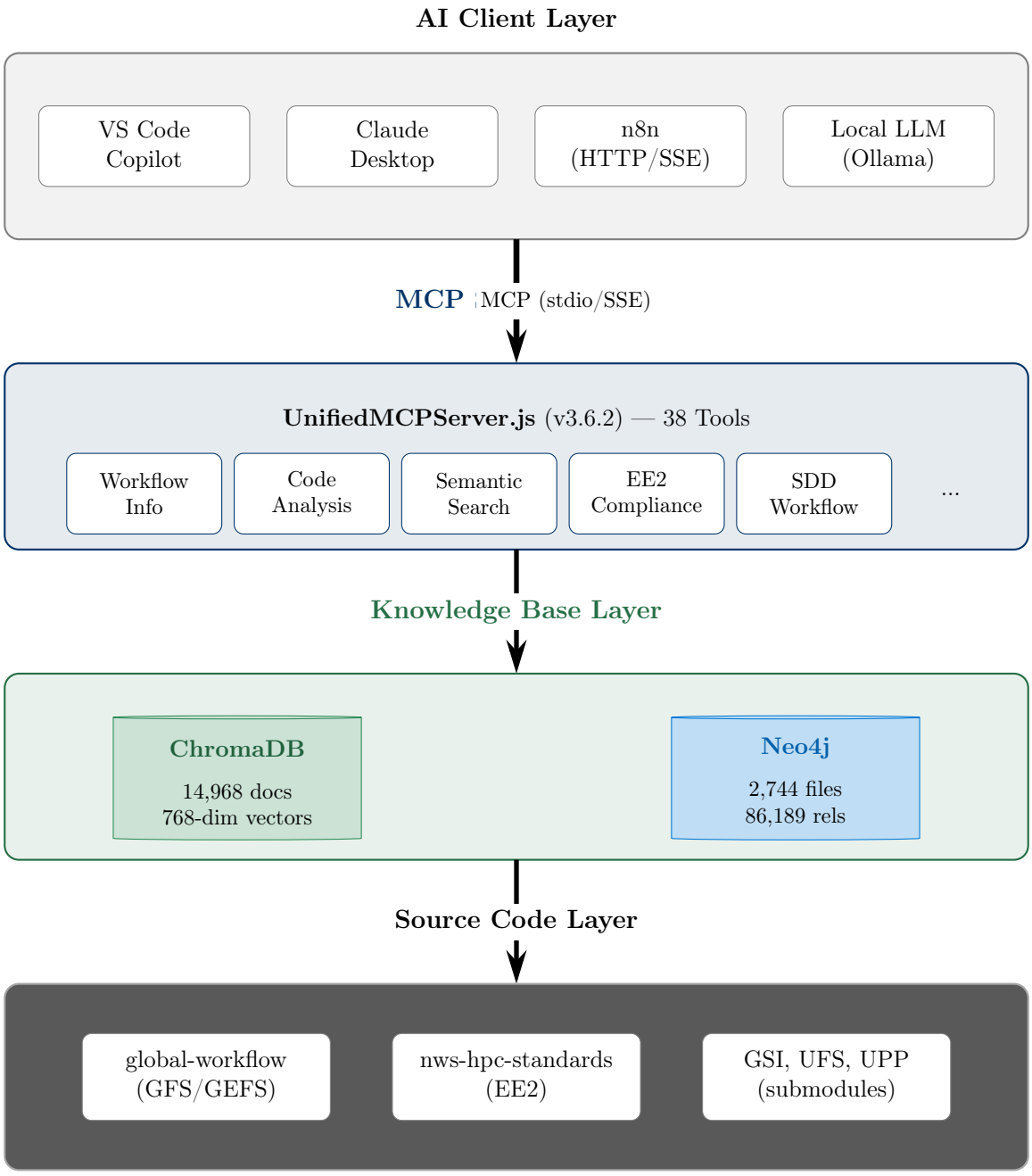


Figure 1: MCP/RAG System Architecture — Four-layer design with COTS LLM independence

2.2 Component Inventory

Component	Technology	Version	Port	Purpose
MCP Server	Node.js	v3.6.2	stdio	Tool orchestration
Vector DB	ChromaDB	0.5.x	8080	Semantic similarity
Graph DB	Neo4j	5.13.0	7474/7687	Code relationships
Embeddings	all-mpnet-base-v2	Latest	—	768-dim vectors
Gateway	Docker MCP	1.0.x	8888	SSE transport
Workflow	n8n	2.1.x	5678	Automation orchestration

Table 2: Core infrastructure components

3 Knowledge Base Design

3.1 ChromaDB Collections

The vector store organizes documents into purpose-specific collections:

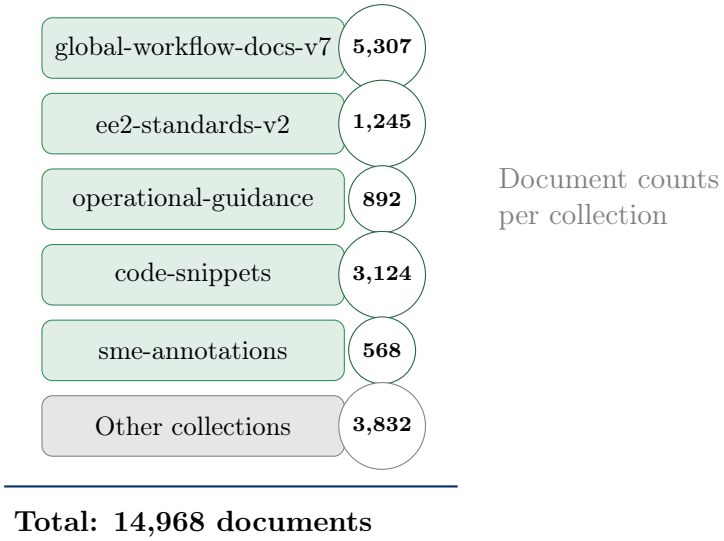


Figure 2: ChromaDB collection organization with document counts

3.2 Neo4j Graph Schema

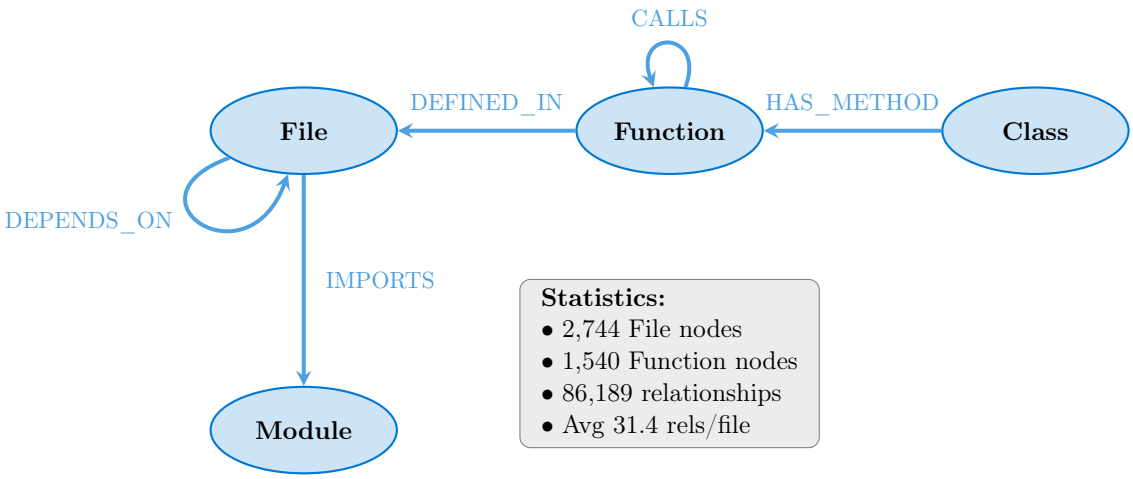


Figure 3: Neo4j graph schema showing node types and relationships

4 Hybrid Retrieval Algorithm

4.1 Scoring Function

The hybrid retrieval combines three components using a weighted sum:

$$\text{hybrid\_score} = \alpha \cdot \text{vector\_score} + \beta \cdot \text{graph\_score} + \gamma \cdot \text{annotation\_score} \quad (1)$$

Where:

- α (vector_score): Cosine similarity from ChromaDB embeddings
- β (graph_score): Structural relevance from Neo4j traversal
- γ (annotation_score): Semantic annotation intent matching

4.2 Query Type Detection and Weight Matrix

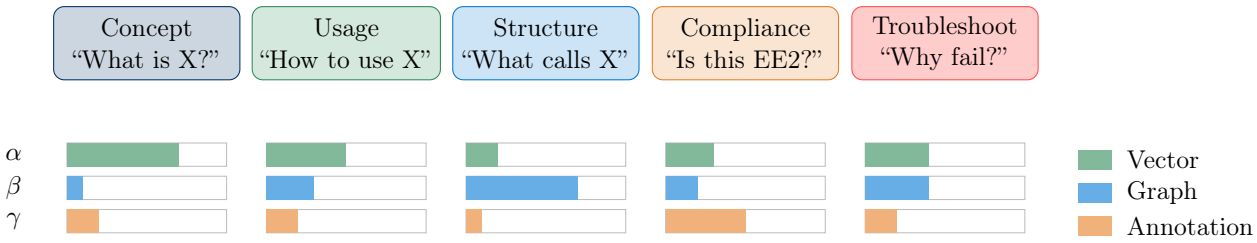


Figure 4: Query type detection adjusts retrieval weights automatically

Query Type	α (vector)	β (graph)	γ (annotation)
Concept	0.7	0.1	0.2
Usage	0.5	0.3	0.2
Structure	0.2	0.7	0.1
Compliance	0.3	0.2	0.5
Troubleshoot	0.4	0.4	0.2

Table 3: Weight matrix for hybrid retrieval by query type

5 Semantic Annotation System

5.1 Overview

The semantic annotation system embeds machine-readable directives in RST documentation that:

- Guide AI compliance analysis with structured metadata
- Prevent false positives via SME corrections
- Provide platform-specific operational guidance
- Remain invisible to human readers (RST comments)

5.2 Directive Types

Directive	Purpose	Example
`mcp:compliance::`	Mark EE2 requirements	`mcp:compliance:: environment_variables`
`mcp:intent::`	Describe enforcement rationale	`mcp:intent:: environment_validation`
`mcp:severity::`	RFC 2119 levels	`mcp:severity:: must`
`mcp:utility::`	Document prod utilities	`mcp:utility:: err_chk`
`mcp:sme_correction::`	Correct AI false positives	`mcp:sme_correction:: bash_error`
`mcp:guidance::`	Platform-specific guidance	`mcp:guidance:: hera_environment`
`mcp:anti_pattern::`	Mark incorrect patterns	`mcp:anti_pattern:: bare_exit`

Table 4: MCP semantic annotation directive types

5.3 SME Corrections — Critical Feature

Warning

SME corrections address systematic AI false positives. The most common example:
AI-Generated Recommendation (INCORRECT):

> ✗ "Missing `set -eu` in scripts"

SME Correction:

- ✗ `set -eu` is **NOT** in EE2 standards
- ✗ `set -e` is **NOT** required in operational scripts
- ✓ Only `set -x` is shown in EE2 examples for debug logging

False Positive Rate: ∼80% of scripts flagged incorrectly without this correction

6 MCP Tool Inventory

6.1 Tool Categories (38 Tools)

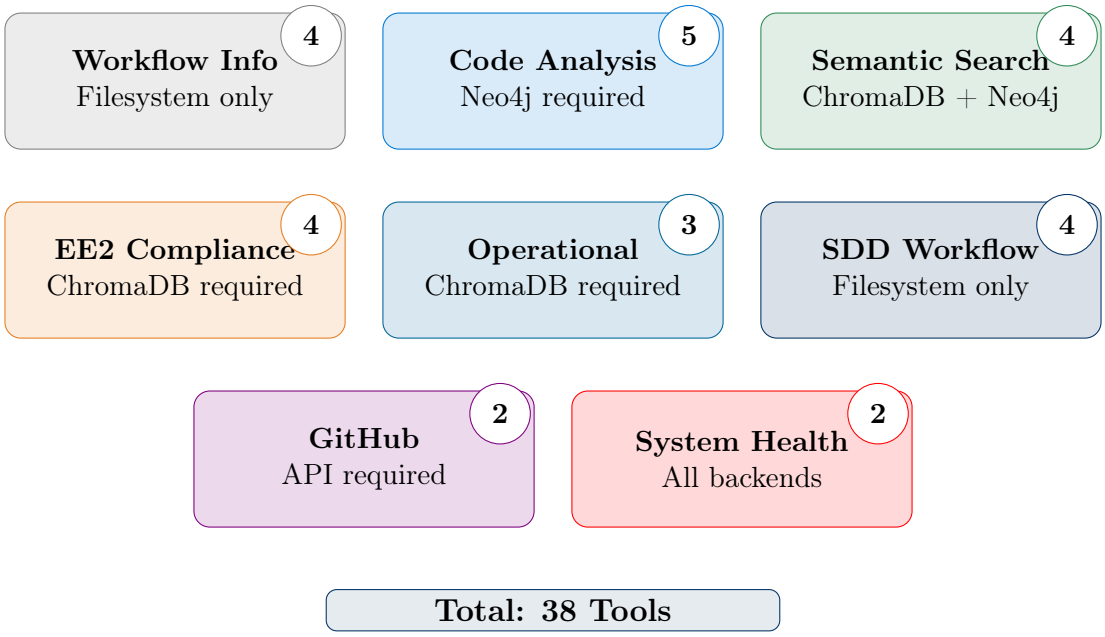


Figure 5: MCP tool categories with backend dependencies

6.2 Tool Selection Decision Tree

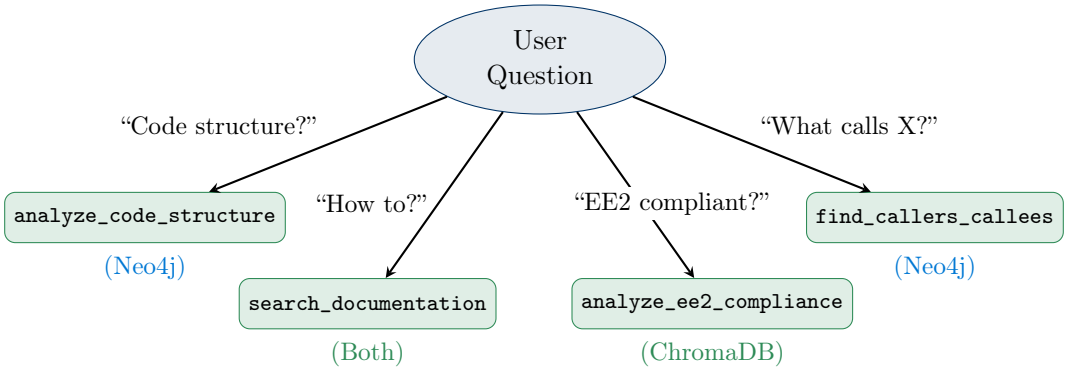


Figure 6: Tool selection based on user question type

7 Configuration Reference

7.1 Environment Variables

```
# MCP Server Configuration
MCP_SERVER_VERSION=3.6.2
MCP_SERVER_MODE=full  # Options: full, core, lite

# Database Connections
CHROMADB_HOST=localhost
CHROMADB_PORT=8080
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=password

# Knowledge Base Paths
MCP_WORKFLOW_ROOT=/mcp_rag_eib/.../supported_repos/global-workflow
KNOWLEDGE_BASE_PATH=/mcp_rag_eib/.../mcp_server_node/knowledge-base

# Performance Tuning
MAX_QUERY_RESULTS=50
SEMANTIC_THRESHOLD=0.1
GRAPH_TRAVERSAL_DEPTH=3
```

Listing 1: mcp-config.env — Primary configuration file

7.2 VS Code MCP Configuration

```
{
  "servers": {
    // GATEWAY MODE: Docker container with full 34 tools via HTTP
    // Transport: Streamable HTTP (MCP spec 2025-06-18)
    "eib-mcp-gateway": {
      "type": "http",
      "url": "http://localhost:18888/mcp",
      "headers": { "Authorization": "Bearer eib-mcp-gateway-token-2025" }
    },
    // LOCAL MODE: Direct stdio for development
    "eib-mcp-rag-full": {
      "type": "stdio",
      "command": "node",
      "args": ["mcp_server_node/src/UnifiedMCPServer.js", "full"],
      "env": {
        "CHROMADB_URL": "http://localhost:8080",
        "NEO4J_URI": "bolt://localhost:7687",
        "NEO4J_PASSWORD": "gfsworkflow2025"
      }
    }
  }
}
```

Listing 2: .vscode/mcp.json — Docker MCP Gateway integration

8 Deployment Procedures

8.1 Fresh Deployment

```
# Step 1: Clone repository
cd /mcp_rag_eib
git clone https://vlab.noaa.gov/.../eib-mcp-rag-server.git
cd eib-mcp-rag-server

# Step 2: Initialize Git submodules
git submodule update --init --recursive

# Step 3: Start infrastructure
docker compose -f docker-compose.devops.yaml up -d

# Step 4: Verify services
curl http://localhost:8080/api/v2/heartbeat  # ChromaDB
curl http://localhost:7474                    # Neo4j

# Step 5: Install dependencies and ingest
cd mcp_server_node && npm install
python3 scripts/ingest_ee2_v7.py
node scripts/build_code_graph.js

# Step 6: Start MCP server
node src/UnifiedMCPServer.js full
```

Listing 3: Complete deployment sequence

8.2 Docker Gateway Deployment

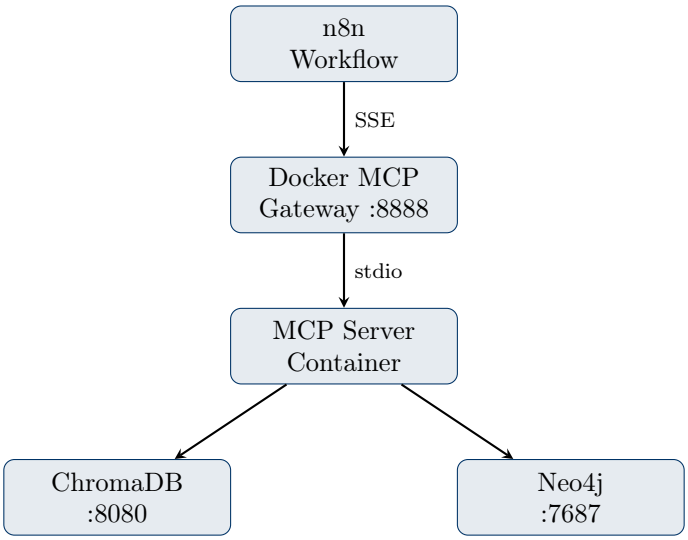


Figure 7: Docker Gateway deployment topology

9 Performance Specifications

9.1 Response Time Targets

Query Type	Target	Typical	Notes
Concept queries	<500ms	350ms	Vector search dominant
Usage queries	<1000ms	650ms	Balanced hybrid
Structure queries	<1500ms	1100ms	Graph traversal overhead
Compliance queries	<800ms	550ms	Annotation matching
Graph traversal	<2000ms	1400ms	Depth-dependent

Table 5: Response time targets by query type

9.2 Resource Requirements

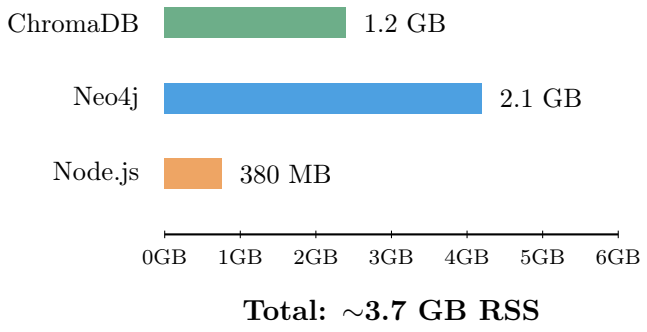


Figure 8: Memory usage by component (production)

Resource Requirements

Minimum:

- 4 CPU cores
- 8 GB RAM
- 20 GB storage

Recommended:

- 8 CPU cores
- 16 GB RAM
- 100 GB storage

10 Troubleshooting Guide

10.1 Common Issues and Solutions

ChromaDB Connection Errors

Symptom: `Connection refused to http://localhost:8080`

```
# Check service status
docker ps | grep chromadb

# View logs
docker logs chromadb --tail 50

# Restart
docker compose -f docker-compose.devops.yaml restart chromadb
```

Neo4j Query Timeouts

Symptom: `Query exceeded 30000 ms limit`

```
# Add indexes in Neo4j Browser
CREATE INDEX function_name_index FOR (f:Function) ON (f.name);
CREATE INDEX file_path_index FOR (f:File) ON (f.path);

# Verify indexes
SHOW INDEXES;
```

MCP Tool "Disabled by User"

Note: This typically means the tool *errored*—not that the user disabled it. This is a VS Code quirk.

```
# Check MCP server logs for actual error
tail -f mcp_server_node/logs/mcp-server.log
```

11 Reproducibility Checklist

11.1 Full System Reproduction

Requirement	Specification	Status
Hardware	8-core CPU, 16GB RAM, 100GB storage	☐
OS	Rocky Linux 8.6+ or Ubuntu 22.04+	☐
Git	Version 2.30+	☐
Docker	Version 20.10+	☐
Node.js	Version 18.0+	☐
Python	Version 3.11+	☐
Network	Access to vlab.noaa.gov, Docker Hub, HF Hub	☐

Table 6: Prerequisites checklist for full system deployment

11.2 Deployment Steps Summary

1. Clone repository from vlab.noaa.gov
2. Initialize submodules: `git submodule update -init -recursive`
3. Start infrastructure: `docker compose -f docker-compose.devops.yaml up -d`
4. Install dependencies: `cd mcp_server_node && npm install`
5. Ingest data: `python3 scripts/ingest_ee2_v7.py`
6. Build graph: `node scripts/build_code_graph.js`
7. Start MCP: `node src/UnifiedMCPServer.js full`
8. Test: `curl http://localhost:8080/api/v2/heartbeat`

Key Insight

Expected deployment time: 2–3 hours (including data ingestion)

A Glossary

Term	Definition
EE2	EMC Environment 2.0 — NCO production coding standards
MCP	Model Context Protocol — AI tool integration standard
RAG	Retrieval-Augmented Generation
GFS	Global Forecast System — NOAA's flagship weather model
GEFS	Global Ensemble Forecast System
HPC	High-Performance Computing platforms (Hera, WCOSS2, etc.)
NCO	NCEP Central Operations — production deployment authority
SDD	Spec-Driven Development — workflow orchestration methodology
SPOT	Single Point of Truth — authoritative configuration source

Table 7: Glossary of terms and acronyms

B Related Documentation

Document	Location
SDD Framework	`sdd_framework/`
Bootstrap Capability	`presentations/papers/sdd_framework/`
Docker Gateway	`presentations/papers/docker_mcp_gateway/`
Copilot Instructions	`.github/copilot-instructions.md`

Table 8: Related documentation locations

Document Control

Version	Date	Author	Changes
1.0	Nov 19, 2025	EIB Team	Initial appendix
2.0	Jan 6, 2026	EIB Team	Elevated to SPOT; comprehensive rewrite

Contact: Terry.McGuinness@noaa.gov

Repository:
https://vlab.noaa.gov/gitlab-community/NWS/Operations/NCEP/EMC/eib-mcp-rag-server